SAMSUNG

Big Data Course

Capstone Project

☒inal Report

For students (instructor review required)

ⓒ2023 SAMSUNG. All rights reserved.
Samsung Electronics Corporate Citizenship Office holds the copyright of this document.
This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this document other than the curriculum of Samsung Innovation Campus, you must receive written consent from copyright holder.

SentimentGuard: Real-Time Sentiment Classification of Live Chat Comments for Early Detection of Harmful Comment Spikes

19/10/26

Team 6

Vàng A Ký (Team Leader)
Nguyễn Đình Bằng
Vàng Thị Dẩm
Dương Thị Hạnh
Ngô Lương Nguyên

Content

1. Introduction

1.1. Background Information

1.2. Motivation and Objective

1.3. Members and Role Assignments

1.4. Schedule and Milestones

2. Project Execution

2.1. Simulated Scenario Description

2.2. Datasets Selection and Description

2.3. Data Ingestion Pipeline

2.4. Data Transformation Processing

2.5. Data Query and Insight

3. Results

3.1. Data Ingestion Scripts and Code

3.2. Data Transformation Scripts and Code

3.3. Description and Sample of Transformed Datasets

3.4. Data Visualization of Query Results

4. Projected Impact

4.1. Accomplishments and Benefits

4.2. Future Improvements

5. Team Member Review and Comment

6. Instructor Review and Comment

1. Introduction

1.1. Background Information

Video platforms and livestreaming services receive a continuous flow of viewer comments. Most messages are harmless, but a sudden rise in hostile or strongly negative comments can damage community trust, drive viewers away, and overwhelm moderators who still read comments manually. The difficulty is not only volume. Harmful messages arrive in bursts, so a warning that appears hours later is no longer useful. A practical moderation system must ingest comments continuously, classify them quickly, and show aggregated signals to a human moderator without collecting unnecessary personal data.

SentimentGuard is the Team 6 capstone prototype built for this problem. It connects Apache Kafka, Spark Structured Streaming, a Spark ML text-classification pipeline, and a Streamlit dashboard in one reproducible workflow. The system was designed as a Technology-for-Good project: it supports safer online communities, gives moderators an early warning, and exposes only aggregate statistics. Because a real livestream feed was not available under the course constraints, public IMDb reviews are replayed through Kafka. The replay stream is a simulation, not real livestream data. Every volume, latency, and alert result in this report must be read in that context.

1.2. Motivation and Objective

The motivation is simple: a moderator cannot read thousands of comments per minute, but a machine can rank and summarise them. The project therefore aims to answer one engineering question: can a small, reproducible Big Data stack detect a negative sentiment spike early enough to support a human decision? The specific objectives were:

- Build a reproducible data-preparation pipeline that cleans text, removes duplicate and near-duplicate comments, and creates fixed train/validation/test splits.
- Train and evaluate a three-class sentiment classifier with Spark ML, using macro-F1 and per-class precision, recall, and F1 instead of accuracy alone.
- Implement a Kafka-to-Spark Structured Streaming pipeline with an explicit schema, event-time processing, a watermark, per-query checkpoints, and a dead-letter queue (DLQ).
- Produce Spark SQL analyses and English-labelled figures that describe class imbalance, comment length, vocabulary, emoji use, throughput, alert behaviour, and latency.
- Provide a Streamlit dashboard and a standalone model-trial program that can be demonstrated without a full cluster.
- Verify that the streaming path produces the same predictions as offline scoring, and document every limitation honestly.

Scope. The project is a prototype, not a production moderation system. It does not collect real livestream comments, does not identify individual users, and does not claim exactly-once delivery; these boundaries are stated again in Section 4.

1.3. Members and Role Assignments

The team has five members. The table below lists the main responsibility of each member; the WBS and Git history contain the detailed task breakdown.

Table *1. Members and role assignments.*

Member	Role	Main contributions
Vàng A Ký	Team Leader	Project planning and topic framing; Action Plan and WBS coordination; presentation preparation; final report integration and submission check.
Nguyễn Đình Bằng	Data ingestion	Kafka 4.3.1 (KRaft) setup and three-topic design; replay producer; bad-event generator; ingestion throughput measurement.

Member	Role	Main contributions
Vàng Thị Dắm	Streaming and text processing	Shared Spark SQL text-cleaning module; Structured Streaming job with explicit schema, event time, watermark, checkpoints, and DLQ; usage scenario and operating guide.
Dương Thị Hạnh	Data and model	Data source and CC0 licence verification; quality assessment, rating mapping, deduplication, fixed split; training, tuning, and evaluation of five models.
Ngô Lương Nguyên	Analytics and dashboard	Thirteen Spark SQL queries; fifteen figures and insight notes; Streamlit dashboard; standalone model-trial program; report figures.
All members	Presentation and demo	Proposal, prototype, and final presentations; report review; live demonstration.

1.4. Schedule and Milestones

The WBS defined a 20-day plan from Day 1 (15/09/2026) to Day 20 (04/10/2026). The implementation sessions recorded in this repository began on 02/10/2026 and were completed on 19/10/2026, after the nominal milestones. Table 2 reports the planned dates and the actual completion dates. It does not claim that the original deadlines were met.

Table *2. Planned and actual schedule.*

Milestone	Planned (WBS)	Actual	Deliverable
Team setup and topic framing	15-16/09/2026	02/10/2026	docs/topic_decision.md
Action Plan	16-17/09/2026	02/10/2026	deliverables/Action_Plan.docx
Preliminary presentation (Day 5)	19/09/2026	14/10/2026	deliverables/Proposal_Slides.pptx
Data preparation	20-22/09/2026	14-15/10/2026	data/processed/*.parquet
Kafka ingestion	22-24/09/2026	14/10/2026	kafka/, src/producer/
Streaming transformation	24-26/09/2026	15/10/2026	src/streaming/stream_job.py
Model training and evaluation	25-28/09/2026	15/10/2026	outputs/models/sentiment_pipeline
Prototype presentation (Day 11)	25/09/2026	15/10/2026	outputs/streaming_summary.md
EDA, figures, and dashboard	28/09-01/10/2026	15/10/2026	src/eda/, src/app/
Final report (Day 18)	02/10/2026	19/10/2026	deliverables/Final_Report.docx
Final presentation and demo (Day 20)	04/10/2026	19/10/2026	Prototype demo and this report

2. Project Execution

2.1. Simulated Scenario Description

The simulated scenario is a moderation desk for a video platform. Each stream_id represents one live room; in the replay, the stream_id is a movie or series title. Viewer comments arrive on the Kafka topic comments_raw as JSON messages with five fields: comment_id, stream_id, user_hash, text, and event_ts. Labels are never sent to Kafka.

A moderator watches the dashboard. The system scores each comment as negative, neutral, or positive, and updates a one-minute tumbling window and a ten-second sliding window. An alert is raised when the negative share is above 30% and the window contains at least five comments. The moderator can then inspect the room, decide whether to slow the chat, pin a message, or escalate. Figure 1 shows the full architecture.

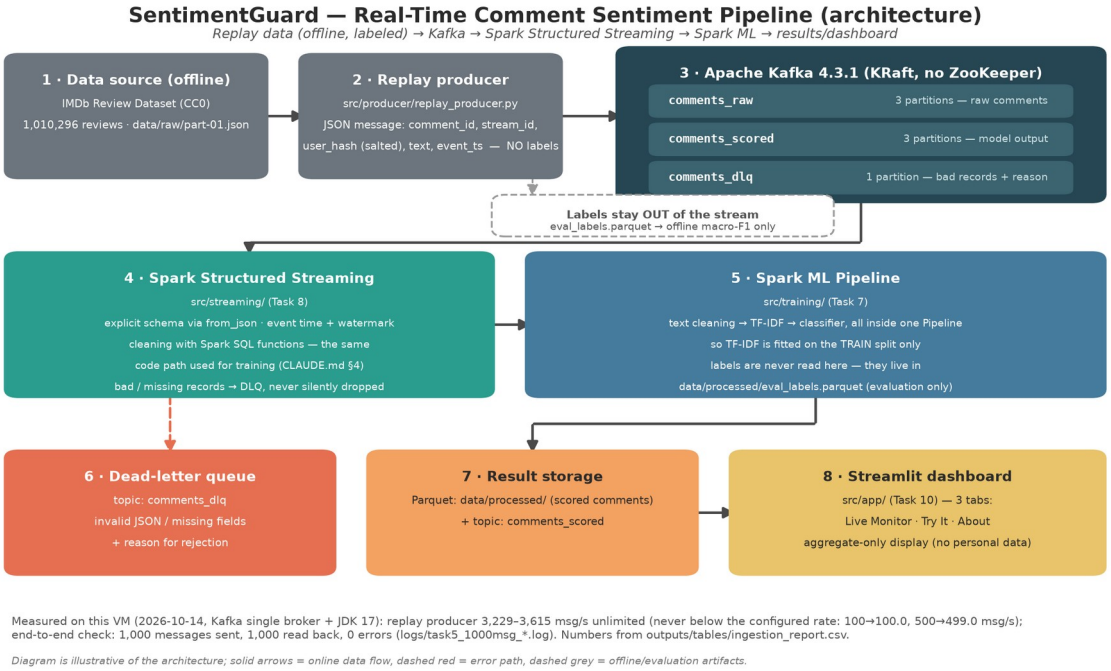


Figure 1. End-to-end architecture: Kafka ingestion, shared cleaning, Spark ML scoring, windowed aggregation, DLQ, Parquet sinks, and the dashboard.

The end-to-end flow has five steps:

- The replay producer publishes five-field JSON messages to comments_raw.
- The streaming job parses the explicit schema, applies the shared cleaning function, and scores valid comments with the saved PipelineModel.
- Valid comments go to scored_detail and to the comments_scored topic; invalid comments go to the DLQ with a reason.
- Two window queries aggregate by stream_id and event time: one-minute tumbling and ten-second sliding windows.
- The Streamlit dashboard reads the Parquet sinks and applies the alert rule; it does not need Spark or Kafka at read time.

The official verification run replayed 1,000 labelled test reviews, 14 valid heartbeat/probe records, and 10 deliberately malformed records. Of the 1,024 records sent, 1,014 were scored and 10 were routed to the DLQ. The 10 errors cover invalid JSON, missing fields, an unparsable timestamp, empty text, text shorter than 20 characters, and text longer than 5,000 characters.

2.2. Datasets Selection and Description

The training data is a 201,074-review shard of the IMDb Review Dataset - ebD, published on Kaggle by Enam Biswas under the CC0 public-domain licence. The source file part-01.json contains 1,010,293 reviews, and the Kaggle page lists 5,571,499 reviews in the complete dataset. The project used a deterministic 19.9% sample because the virtual machine has limited RAM and disk; this is a subset, not the complete corpus.

The source has no sentiment label. The team defined a three-class proxy label from the rating: 1-4 negative, 5-6 neutral, and 7-10 positive. This mapping is a team convention, not an official IMDb label. It is the main source of label noise, especially for the neutral class.

Table *3. Source fields and their use in the project.*

Source field	Project field	Use
review_id	comment_id	Join key across Parquet, Kafka, and evaluation labels.
movie	stream_id	Simulated room; also the grouping key for the split.
reviewer	user_hash	Salted SHA-256; the original value is not stored.
review_detail	text	Comment text used by the model.
rating	label, rating	Proxy label (1-4 / 5-6 / 7-10); rating kept for traceability.
review_date	review_date	Reference only; not used as event time.
review_summary, spoiler_tag, helpful	(dropped)	Not needed for the simulation; not stored.

Privacy. The reviewer field is hashed with a salt read from the COMMENT_HASH_SALT environment variable. Only the first 16 hexadecimal characters of the hash are stored. The dashboard shows aggregates, not individual users. The original reviewer name or ID never appears in the processed Parquet files.

2.3. Data Ingestion Pipeline

Kafka 4.3.1 runs in KRaft mode, without ZooKeeper. Kafka 4.x requires Java 17, while Spark 3.5.9 in this environment uses Java 8; the two runtimes are installed separately. Table 4 lists the three topics.

Table *4. Kafka topics used by the pipeline.*

Topic	Partitions	Purpose
comments_raw	3	Raw five-field JSON from the replay producer.
comments_scored	3	Scored JSON from the streaming job.
comments_dlq	1	Invalid records with a machine-readable reason.

The replay producer reads the fixed test split and publishes one JSON message per comment. It preserves a deterministic order with seed 42, supports a target rate and burst mode, and can deliberately delay a fraction of records. In the official run the target rate was 50 messages/s and the burst ceiling was 400 messages/s. The producer waits for broker acknowledgements and records delivery failures.

Ingestion was measured on the single virtual machine. 969 of the 1,014 scored comments arrived within 27 seconds, and the highest one-second rate was 384 messages/s, which is 96% of the 400 messages/s design ceiling. This is a replay-load result, not a capacity claim for a production cluster.

2.4. Data Transformation Processing

Data preparation is a separate batch step (src/training/prepare.py) so that training and streaming share the same cleaning logic. The raw JSON array is converted to JSONL for parallel reading, sampled deterministically, cleaned with Spark SQL functions, deduplicated in three passes, and split by movie into 70/15/15 train/validation/test sets with seed 42. Table 5 records every removal.

Table *5. Data-preparation log (outputs/tables/cleaning_ log.csv).*

Step	Rows in	Removed	Rows out	Reason
01 Load raw	201,074	0	201,074	JSONL shard
02 Empty text	201,074	0	201,074	No empty text after cleaning
03 Missing label	201,074	10,394	190,680	Rating missing or outside 1-10
04 Too short	190,680	10	190,670	Fewer than 20 characters
05 Too long	190,670	1,686	188,984	More than 5,000 characters
06 Duplicate ID	188,984	2,999	185,985	Keep the longest text
07 Exact duplicate	185,985	636	185,349	Identical cleaned text
08 Near duplicate	185,349	63	185,286	Same token set after normalisation
09 Remaining	185,286	0	185,286	Used for the fixed split

The cleaning function is written once in src/common/text_cleaning.py and is used by both training and streaming. It lowercases, removes HTML, replaces URLs with <url> and mentions with <user>, normalises punctuation, expands abbreviations, reduces repeated characters, separates emoji into tokens, and collapses whitespace. All steps are Spark SQL expressions; no Python UDF is used. The function is idempotent, so cleaning an already-cleaned comment does not change it.

The split is grouped by movie, not by row, to prevent the same title from appearing in two sets. The three sets have the same class distribution within 0.02 percentage points. The test set was scored once, on 15/10/2026 at 00:05:02, and the event is recorded in outputs/tables/test_eval_log.csv. TF-IDF is fitted inside the Spark ML Pipeline, so the vectorizer sees only training data.

Streaming transformation. The streaming job (src/streaming/stream_job.py) starts six independent queries. Each query has its own checkpoint directory. The job declares the JSON schema explicitly, converts event_ts to an event-time timestamp, applies the shared cleaning function, and routes every record to exactly one branch: valid or DLQ. Table 6 lists the queries.

Table *6. The six streaming queries.*

Query	Output	Purpose
detail	Parquet scored_detail	One row per scored comment, partitioned by event date/hour.
kafka	comments_scored topic	Scored JSON for downstream consumers.
window	window_1min Parquet	One-minute tumbling aggregates per stream_id.
sliding	window_sliding_10s Parquet	Ten-second windows with a five-second slide.
dlq_kafka	comments_dlq topic	Invalid records with a reason.
dlq_parquet	dlq Parquet	Durable copy of the dead-letter queue.

The watermark is two minutes on event_time. Window queries use append mode and wait for the watermark before emitting a window. A record that arrives after the watermark is dropped by design; a record that is malformed or fails validation goes to the DLQ, not to the windows. This distinction matters: late data and bad data are different problems.

Verification. The run sent 1,024 records and accounted for all of them: 1,014 in scored_detail and 10 in the DLQ. The comments_scored topic grew by exactly 1,014 messages and comments_dlq by exactly 10. Re-scoring the same 1,000 test comment_ids offline with the saved PipelineModel produced 1,000 identical predictions (100%). The streaming macro-F1 on that 1,000-comment sample was 0.6311, compared with 0.6233 on the full 27,792-row test set. The difference comes from the sample composition, not from the streaming code path.

Delivery guarantee. The observed run had no duplicate and no loss: 1,014 + 10 = 1,024. The mechanism is at-least-once, because offsets are committed after a micro-batch is written; a crash between write and commit can replay the batch. Exactly-once has not been verified by a crash/restart experiment, and this report does not claim it.

2.5. Data Query and Insight

Thirteen Spark SQL queries in src/eda/sql/ produce the tables behind the figures. They cover dataset size, class distribution, length quantiles and histograms, distinctive words, emoji, sentiment over time, throughput, alerting, latency, and vocabulary size. Table 7 lists them.

Table *7. Spark SQL queries used for the analysis.*

Query	Question	Output
q00 overview	One-row sanity check of the analysis dataset	Console table
q01 class distribution	Class balance in train/validation/test	Figure 6
q02 length quantiles	Comment length by class	Figure 7
q03 length histogram	Length distribution by class	Figure 7
q04 top words	Most distinctive words per class	Figure 8
q05 emoji	Emoji use per 1,000 comments by class	Figure 9
q06 sentiment per minute	One-minute sentiment timeline	Figure 10 (top)
q07 sentiment per 10s	Ten-second sliding timeline	Figure 10 (bottom)
q08 throughput	Messages per second and burst flags	Figure 11
q09 negative share alert	Negative share and alert rule	Figure 12
q10 latency quantiles	p50/p95/p99 end-to-end latency	Figure 13
q11 latency histogram	Latency distribution buckets	Figure 13
q12 token vocabulary	Vocabulary growth from punctuation tokens	Table only

The main findings are:

- The data is imbalanced: positive 61.1%, negative 25.1%, neutral 13.9%. A majority-class baseline already reaches 61.1% accuracy, so accuracy alone is misleading; macro-F1 is the primary metric.
- Neutral comments are the longest (median 131 tokens) and the hardest class (test F1 0.378). Positive comments have a median of 83 tokens.
- The replay burst reached 384 messages/s, 96% of the 400 messages/s design ceiling. 969 of 1,014 comments arrived in 27 seconds.
- The alert rule fired once, at 18:08:30 UTC, with a 38.5% negative share and 13 comments. Removing the minimum of five comments would add 13 false alarms.
- End-to-end latency has two peaks: 81% of comments were scored in 1.0-4.0 s (p50 2,413 ms), but p95 was 9,663 ms and p99 was 27,091 ms. Queueing inside the micro-batch dominates; Kafka transfer is effectively local (p50 0 ms).
- Tokenisation is coarse: 11.2% of tokens carry punctuation, and the vocabulary grows from 318,980 to 524,107 tokens (+64%). This is a known limitation.
- Emoji are weak features: positive comments use 46 emoji per 1,000 comments, neutral 18; the top emoji in the negative class is the face with tears of joy, which is often sarcastic.

All streaming findings describe the replay run. They illustrate the mechanics of the pipeline; they are not claims about real viewer behaviour.

3. Results

3.1. Data Ingestion Scripts and Code

The ingestion code is split into three small scripts so that each step can be run and checked independently.

- kafka/start_kafka.sh, stop_kafka.sh, create_topics.sh: start the KRaft broker, create the three topics, and stop the broker.
- src/producer/replay_producer.py: read the fixed test split, build the five-field JSON message, and publish to comments_raw with rate, burst, and late-record controls.
- scripts/send_bad_events.py: send ten known malformed records plus valid heartbeat and late-probe records for DLQ and watermark verification.

A producer message has exactly five fields and never contains the label:

```
msg = {
    "comment_id": comment_id,
    "stream_id": stream_id,
    "user_hash": salted_hash(reviewer, salt=salt),
    "text": review_text,
    "event_ts": iso_utc(time.time()),
}
producer.send("comments_raw", key=comment_id,
              value=json.dumps(msg).encode("utf-8"))
```

Topics are created with explicit partition counts:

```
kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --if-not-exists --topic comments_raw \
  --partitions 3 --replication-factor 1
```

The full command used in the verification run was: replay_producer.py --only-test --max 1000 --rate 50, followed by send_bad_events.py --heartbeat 12 --heartbeat-interval 25 --late-probe. The producer reported 1,000 delivered messages and 0 send failures.

3.2. Data Transformation Scripts and Code

The training script builds one Spark ML Pipeline for every candidate model. The pipeline is fitted only on the training split; validation is used to compare models, and the test split is touched once at the end.

Table *8. Spark ML Pipeline stages.*

Stage	Configuration	Purpose
RegexTokenizer	Whitespace pattern; text already lowercased	Split cleaned text into tokens.
StopWordsRemover	162 English stop words; negations kept	Remove low-information words without losing 'not good'.
NGram	n=1 (n=2 tested for NB and LR)	Unigrams won in both tested models.
CountVectorizer	vocabSize 20,000; minDF 3	Term-frequency vector.
IDF	Default	Down-weight common terms.
Normalizer	L2	Stabilise linear models.
Classifier	LR / NB / SVC / RF	Compare five models; LR selected.

Class imbalance is handled with weights w_c = N / (k * n_c), computed on the training split only. The weights were negative 1.330, neutral 2.404, and positive 0.546. Hyperparameters were selected with TrainValidationSplit (80/20 inside train) using weighted F1; the validation and test sets were never used for tuning.

Table 9 compares four classifiers and a majority baseline on validation. Logistic Regression had the best macro-F1 (0.6291), trained in 245 seconds, and provides per-class probabilities, which the dashboard and the model-trial program use.

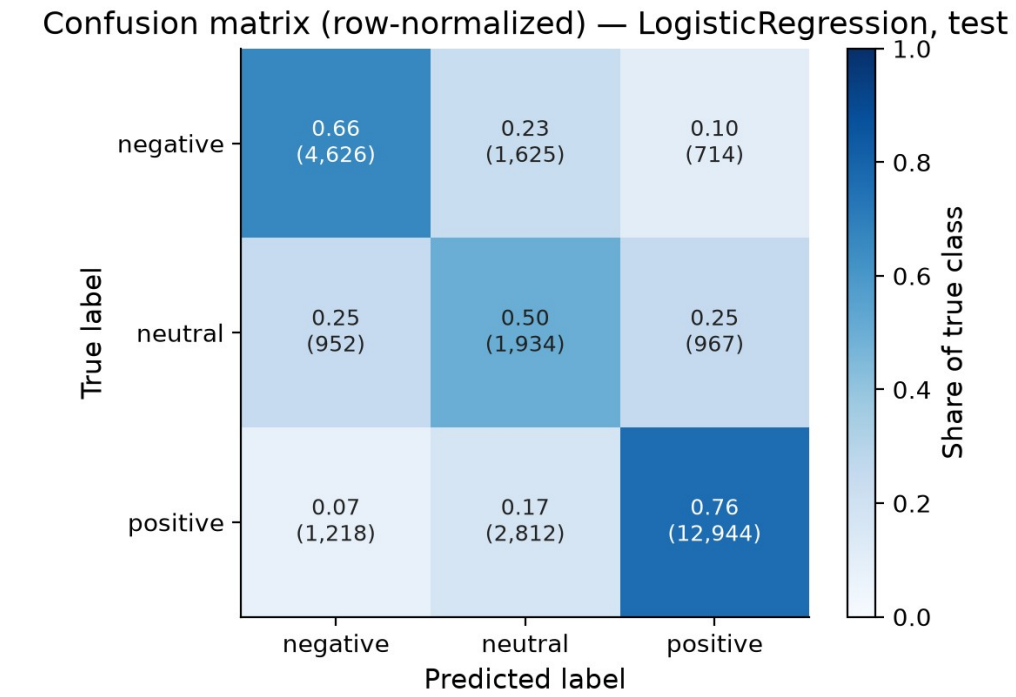
Table *9. Model comparison on the validation split (27,793 rows).*

Model	Accuracy	Weighted F1	Macro-F1	Train time (s)	Features
MajorityBaseline	0.6107	0.4631	0.2528	0	0
NaiveBayes	0.6743	0.7013	0.6192	107.9	20,000
LogisticRegression	0.7063	0.7241	0.6291	245.0	20,000
LinearSVC (OvR)	0.7000	0.7172	0.6218	116.3	20,000
RandomForest	0.5878	0.6131	0.5343	282.4	5,000

The selected model was evaluated once on the test set: accuracy 0.7018, weighted F1 0.7213, macro-F1 0.6233. Table 10 gives per-class metrics, and Figure 2 shows the confusion matrix. The neutral class is the weak point: only 50.2% of neutral comments were recalled, and errors split almost evenly between negative and positive. Positive comments were classified well (F1 0.8193), while 16.6% of positive comments were predicted as neutral.

Table *10. Final test metrics by class (27,792 rows).*

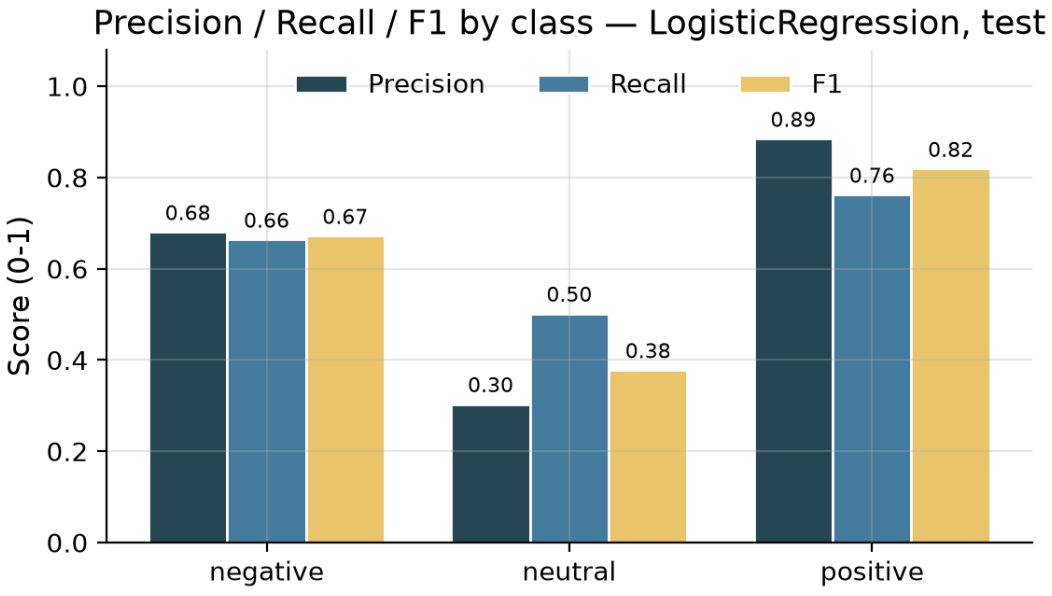
Class	Precision	Recall	F1	Support	Share
negative	0.6807	0.6642	0.6723	6,965	25.06%
neutral	0.3036	0.5019	0.3783	3,853	13.86%
positive	0.8851	0.7626	0.8193	16,974	61.08%



Source: outputs/tables/confusion_matrix.csv — LogisticRegression on test (Task 7)

Figure 2. Confusion matrix on the test set (rows: true class; columns: predicted class).

The neutral row is split almost evenly across all three predictions. This is the expected consequence of a proxy label built from ratings 5-6: those reviews contain both mild praise and mild criticism.



Source: outputs/tables/per_class_metrics.csv — LogisticRegression on test

Figure 3. Precision, recall, and F1 by class on the test set.

Positive has high precision (0.885) but lower recall (0.763); negative is balanced at about 0.67; neutral is below 0.4 on every measure.

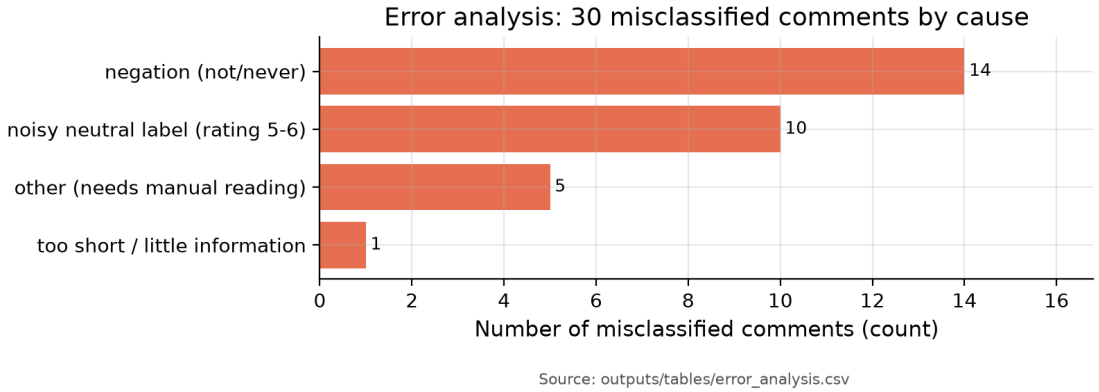


Figure 4. Manual error categories among misclassified test comments.

The largest error categories are noisy neutral labels, suspected sarcasm, and negation or contrast constructions such as 'not good' and 'but'. These are language problems, not streaming problems.

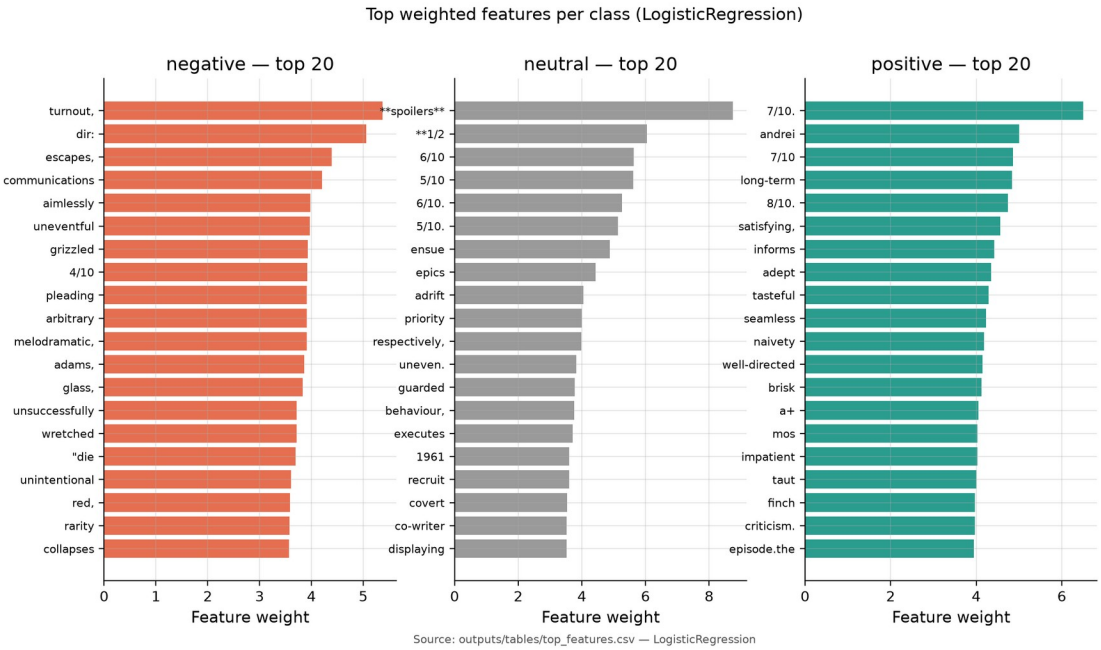


Figure 5. Top weighted TF-IDF features for each class (Logistic Regression).

The top neutral features include explicit rating strings such as '6/10'. The model is partly reading the number in the review, a shortcut that will not exist in most real livestream comments.

Leak checks. No cleaned text appears in more than one split (0 overlaps in train/validation, train/test, and validation/test). The test evaluation log contains exactly one entry. A residual risk remains: the split is grouped by movie, but the same reviewer can review several movies. A post-hoc measurement found 11,427 of 111,087 user hashes (10.3%) in more than one split, covering 75,939 of 185,286 rows (41.0%). The effect on the test metrics was not quantified; it is reported as a limitation rather than treated as evidence of a clean split.

3.3. Description and Sample of Transformed Datasets

The processed data is stored as three Parquet datasets with the same schema. Table 11 describes the columns; Table 12 gives the row counts and class shares.

Table *11. Schema of train.parquet, val.parquet, and test.parquet.*

Column	Type	Meaning
comment_id	string	review_id of the source review; key for labels and Kafka.
stream_id	string	Movie title used as the simulated room.
user_hash	string(16)	First 16 hex characters of salted SHA-256(reviewer).
label	string	negative / neutral / positive (proxy from rating).
rating	string	Original rating 1-10; kept for traceability, not a feature.
text	string	Cleaned text used by the model.
review_date	string	Original review date; reference only.
n_chars	int	Character count of cleaned text.
n_tokens	int	Whitespace token count.

Table *12. Split sizes and class distribution.*

Split	Negative	Neutral	Positive	Total	Share of all rows
Train	32,504	17,983	79,214	129,701	70.0%
Validation	6,965	3,854	16,974	27,793	15.0%
Test	6,965	3,853	16,974	27,792	15.0%
All	46,434	25,690	113,162	185,286	100.0%

Table 13 shows four real rows from outputs/tables/clean_sample.csv. The user_hash is a salted hash; the text is shown as cleaned by the shared function. Long comments are truncated for display only.

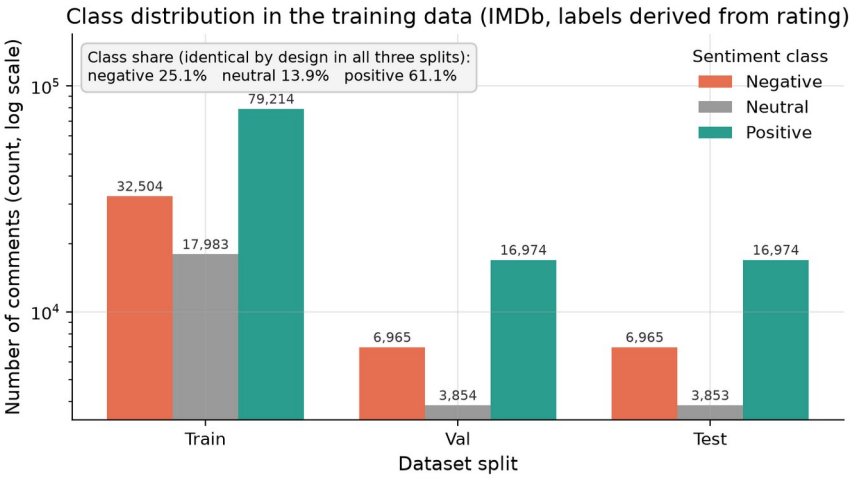
Table *13. Sample rows from the transformed dataset.*

comment_id	stream_id	user_hash	label	rating	split	text (truncated)
rw6358464	The Call (2020)	e87642b1	positive	7	test	good acting, plot was great & overall pretty good movie! imagine the movie frequency, just darker.. recommended
rw5571862	Onward (I) (2020)	3fc79249	positive	8	train	onward is a pretty good animation movie which deserves your time. it's funny, emotional and have some pretty good animated effects.
rw5739267	The Hollow (2018-2020)	fc0564f8	neutral	6	train	at first it seemed not well thought. but as episodes keeps going, they get entertaining. it has no main plot, but it keeps you watching.
rw2994146	Love It or List It (2008-)	ab2492c5	negative	1	train	this could be a great show. i have tried to get through it because i love the idea of seeing the finished product and what they choose. however, the soul crushing stupidity of the fake drama stops me.

The evaluation labels are stored separately in data/processed/eval_labels.parquet for the test split only. The producer never reads labels for the messages it sends; the verification script joins them by comment_id after the run. This separation is what makes the streaming evaluation honest.

3.4. Data Visualization of Query Results

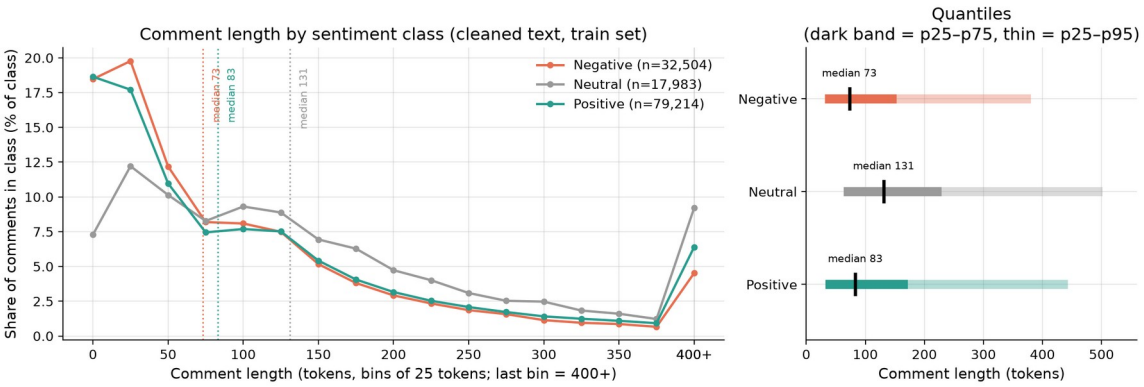
This section presents the figures produced from the Spark SQL tables. Each caption states what is plotted; the paragraph below each figure gives the measured insight. The underlying CSV files are in outputs/tables/.



Source: outputs/tables/eda_class_distribution.csv (Spark SQL q01). Labels are derived from rating 1-4/5-6/7-10 (design choice, not human annotations).

Figure 6. Class distribution in the training data (129,701 rows).

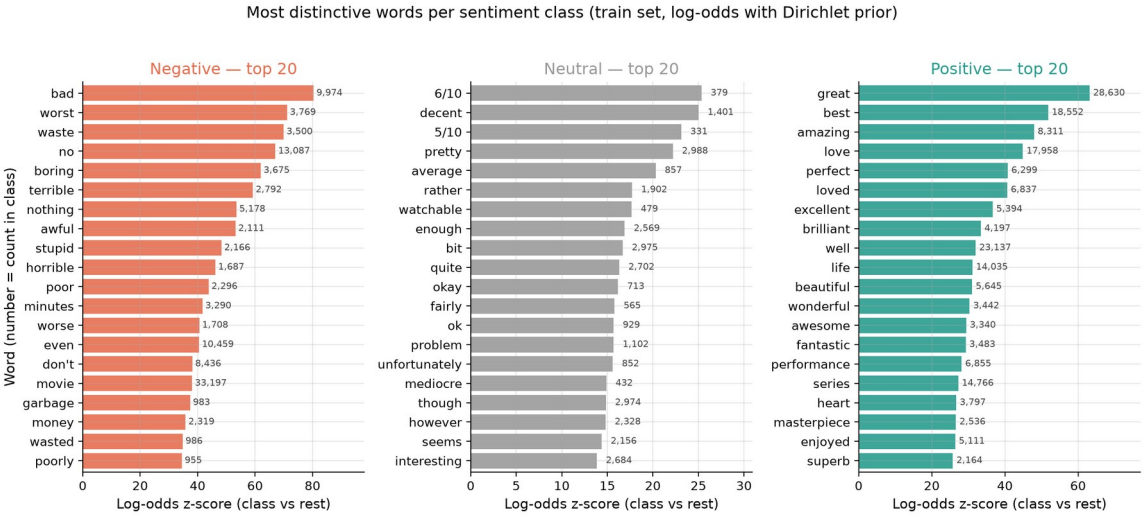
Positive comments dominate (61.1%), negative is 25.1%, and neutral is only 13.9%. The distribution is almost identical in train, validation, and test, so a majority baseline reaches 61.1% accuracy; macro-F1 is the fair metric.



Source: outputs/tables/eda_length_histogram.csv + eda_length_quantiles.csv (Spark SQL q02–q03). 'Tokens' = whitespace split of the cleaned text, same as the model's tokenizer.

Figure 7. Comment length by sentiment class (tokens).

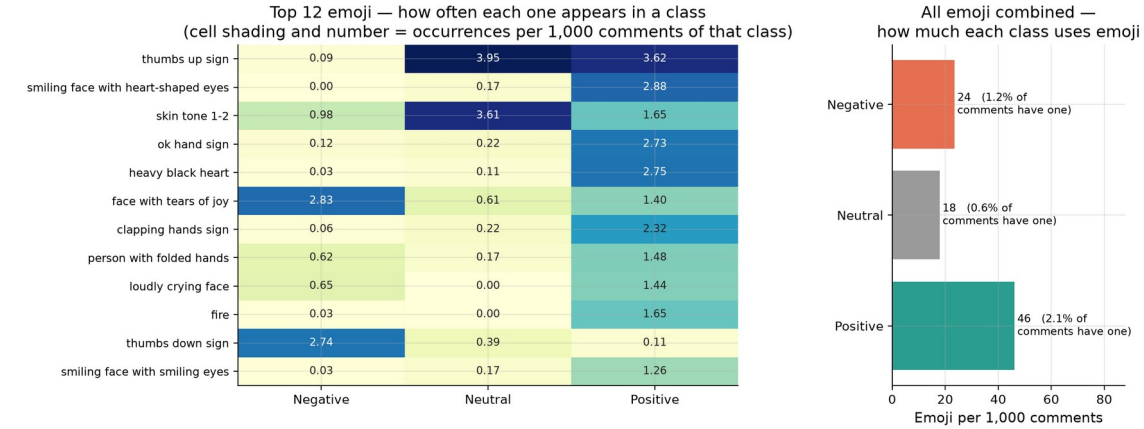
Neutral comments are the longest (median 131 tokens, p95 501); positive comments have a median of 83 tokens. All classes have a long right tail, so comment length varies by more than an order of magnitude.



Source: outputs/tables/eda_top_words_by_class.csv (Spark SQL q04). Frequency-based analysis, NOT the model's learned weights (see Task 7, top_features.csv).

Figure 8. Top 20 distinctive words per class.

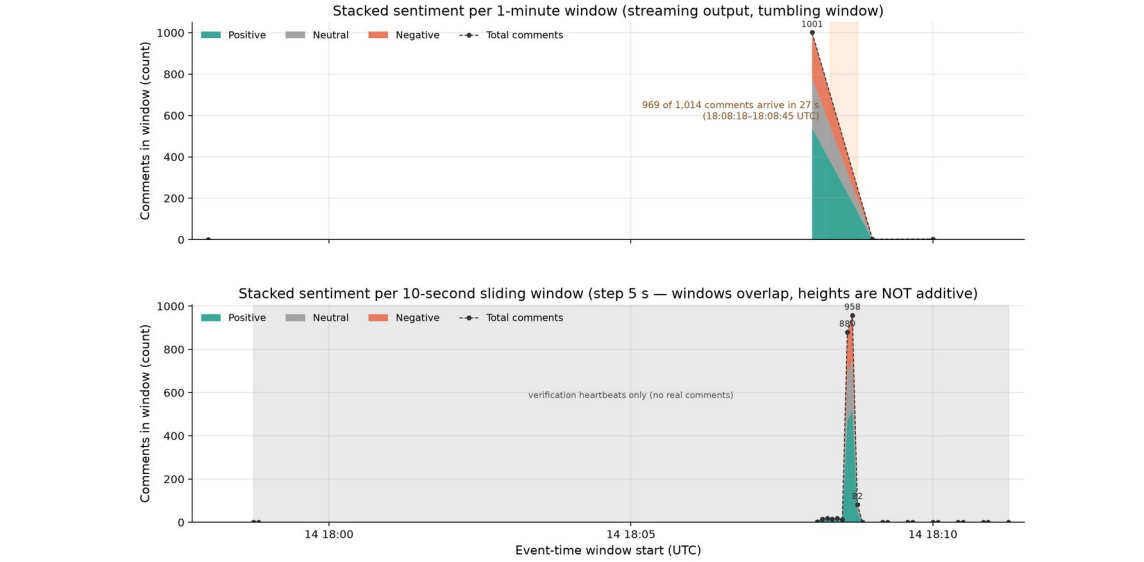
The strongest words are 'bad' for negative, '6/10' for neutral, and 'great' for positive. Two of the 20 neutral markers are explicit rating strings such as '6/10'; the model is partly learning the number in the review, not only sentiment.



Source: outputs/tables/eda_emoji_by_class.csv (Spark SQL q05). Uses text_cleaning.EMOJI_SYMBOL_RANGES, which excludes typographic symbols (e.g. U+00B0) that the cleaner's broader \p{So} class also catches.

Figure 9. Emoji use per 1,000 comments by class.

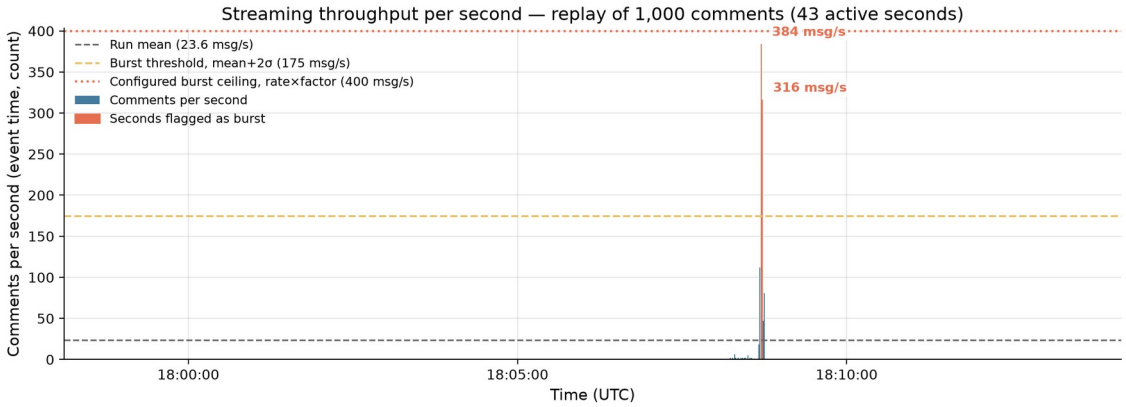
Positive comments use 46 emoji per 1,000 comments, versus 18 for neutral. The absolute rate is low (2.1% of positive comments contain an emoji). The top emoji in the negative class is the face with tears of joy, so emoji cannot be used as a simple positive rule.



Source: outputs/tables/eda_sentiment_per_minute.csv + eda_sentiment_per_10s.csv (streaming window queries of run 'verify1', Spark SQL q06–q07). REPLAY data (IMDb), not a real livestream. Gaps = windows with no comments (no line interpolated across them).

Figure 10. Sentiment over time: one-minute windows (top) and ten-second sliding windows (bottom).

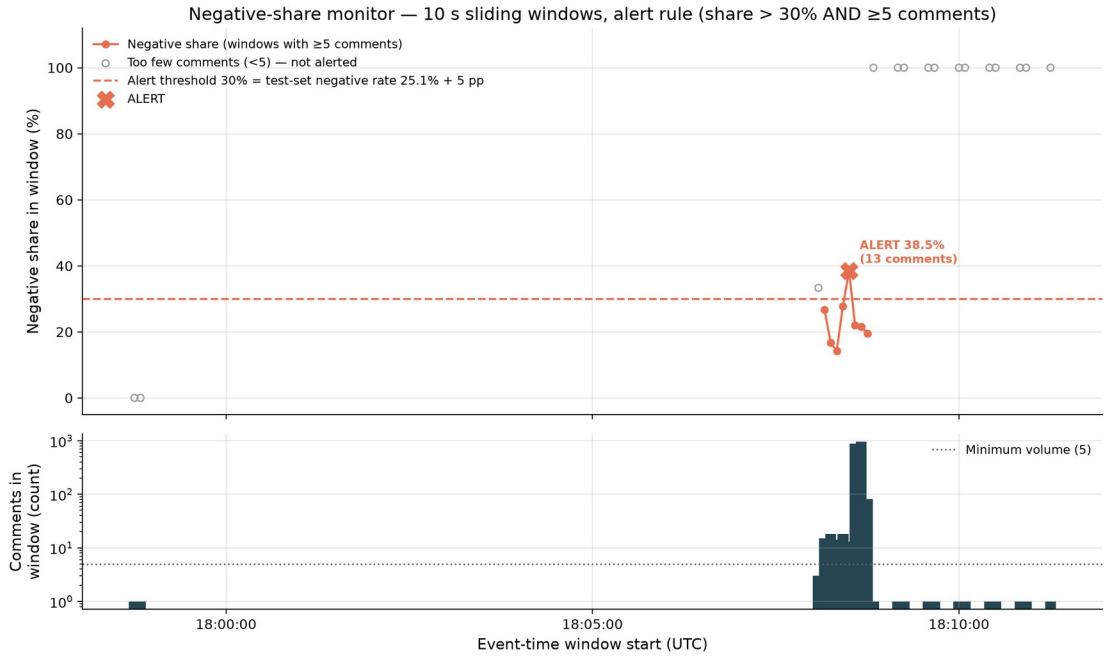
The replay burst is compressed into 27 seconds, so the one-minute view has only four points. The ten-second view shows the negative share moving between 14.3% and 38.5%; this is why the dashboard uses the sliding window.



Source: outputs/tables/eda_throughput_per_second.csv (Spark SQL q08) — burst rule: count > mean + 2σ of this run; ceiling = replay.rate × replay.burst_factor from config.yaml.

Figure 11. Messages per second and burst threshold in the replay run.

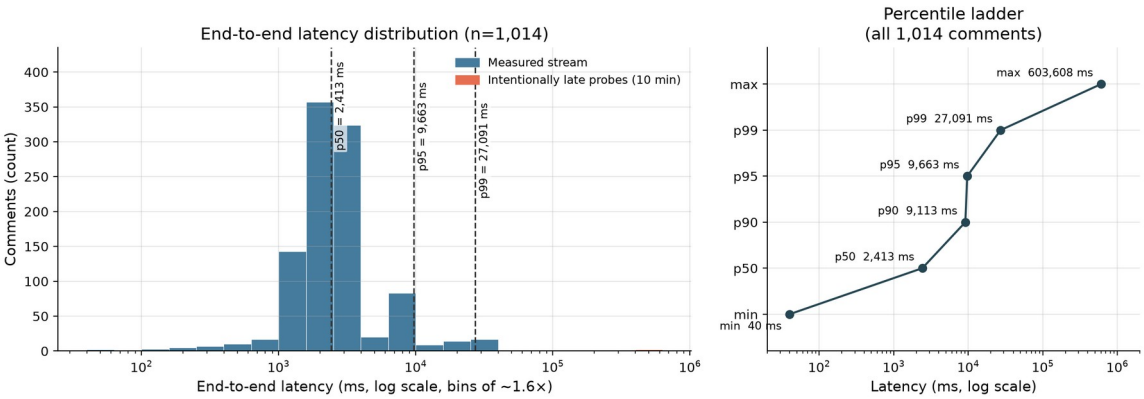
Only 2 of 43 active seconds exceed the 175 messages/s burst threshold: 384 and 316 messages/s. The peak is 96% of the 400 messages/s design ceiling, which is the highest load actually tested.



Source: outputs/tables/eda_negative_share_alert.csv (Spark SQL q09). Replay data (IMDb), not a real livestream; thresholds are configurable in config.yaml (eda.alert).

Figure 12. Negative-share monitor and alert rule.

The rule (negative share › 30% and at least 5 comments) fired once: 38.5% with 13 comments at 18:08:30 UTC. Without the minimum of five comments, the same data would produce 13 false alarms from windows with 1-3 comments.



Source: outputs/tables/eda_latency_histogram.csv + eda_latency_quantiles.csv (Spark SQL q10–q11). Latency = processing time − event time, measured per micro-batch (see src/streaming/README.md §7).

Figure 13. End-to-end latency distribution (log scale).

81% of comments were scored in 1.0-4.0 s, with a median of 2,413 ms. The p95 is 9,663 ms and the p99 is 27,091 ms; the long tail comes from queueing inside the micro-batch, not from Kafka. Two deliberately late records appear at about 603 s.

4. Projected Impact

4.1. Accomplishments and Benefits

The project delivered a complete, reproducible prototype rather than a slide-only design. Table 14 lists the main accomplishments and the file that evidences each one.

Table *14. Main accomplishments and evidence.*

Accomplishment	Evidence
Reproducible data pipeline: 201,074 raw reviews to 185,286 clean rows	outputs/tables/cleaning_log.csv
Fixed grouped split with no duplicate text across sets	outputs/tables/class_distribution.csv; outputs/tables/leak_check.json
Four classifiers plus a majority baseline, and a saved Spark ML Pipeline	outputs/tables/model_results.csv; outputs/models/sentiment_pipeline
Final test evaluation: accuracy 0.7018, macro-F1 0.6233	outputs/tables/test_eval_log.csv; outputs/tables/per_class_metrics.csv
Kafka replay ingestion: 1,024 records sent, 1,014 scored, 10 DLQ	outputs/streaming_summary.md
Streaming/offline agreement: 1,000/1,000 identical predictions	outputs/stream/verify1/verification.json
Thirteen SQL analyses and fifteen figures	src/eda/sql/; outputs/figures/
Streamlit dashboard with a tested alert rule	src/app/app.py; tests/test_dashboard.py
Standalone model-trial program	src/app/model_trial.py; outputs/model_trial_report.md
77 automated tests passed	tests/

Benefits. For a community team, the system provides an early-warning signal instead of a manual backlog. It ranks rooms by negative share, requires a minimum sample before alerting, and shows the number of comments behind every number. For engineers, the repository is a reusable template: shared cleaning code, explicit schemas, per-query checkpoints, a DLQ, and a written verification protocol. For the team, the project demonstrates the full Big Data workflow from ingestion to a user-facing dashboard.

Technology for Good. The project is designed to protect online communities. It does not identify or score individual users publicly, stores only salted hashes, and displays aggregate statistics. The intended use is moderator support, not surveillance.

Limitations. Table 15 lists the limitations that a reader must keep in mind. None of them is hidden in the results.

Table *15. Limitations of the current prototype.*

Limitation	Consequence
Replay, not real livestream	Volume, latency, and alert results are a simulation.
Labels inferred from rating	Neutral is noisy; the model learns rating language.
English movie reviews, not live chat	Domain mismatch; Vietnamese and teencode were not tested.
Rating strings in text ('7/10')	Metrics are optimistic for comments without ratings.
Neutral F1 0.378	One class is not reliably usable without human review.
p95 latency 9.7 s	Not suitable for sub-second moderation.
Single VM, local[2], 1,024 records	No scale or multi-broker evidence.
At-least-once only	Exactly-once not verified; a crash can duplicate a batch.
11,427 users across splits	Residual leakage risk; the effect was not quantified.

4.2. Future Improvements

The next version should address the limitations in the order below.

- Collect real comments with platform permission and an appropriate licence. Start with 200-300 Vietnamese comments labelled by two annotators; measure inter-annotator agreement before training.
- Replace the rating proxy with human labels for neutral and sarcasm. Add active learning so moderators can correct uncertain cases, and keep a held-out set that is never used for tuning.
- Improve the text pipeline: Vietnamese word segmentation, punctuation-aware tokenisation, emoji and teencode handling, and negation/sarcasm features. Measure the cost of any Python UDF before adopting it.
- Evaluate transformer models such as PhoBERT or a multilingual BERT on GPU. Compare macro-F1, latency, and cost with the current TF-IDF + Logistic Regression baseline; do not adopt a heavier model without a measured gain.
- Harden the streaming layer: a multi-broker Kafka cluster, more partitions, a 1-2 s trigger, backpressure, schema registry, and a crash/restart test that verifies exactly-once or documents the duplicate bound.
- Build a real moderator workflow: authentication, alert routing, acknowledgement, feedback buttons, and a drift monitor that compares live class shares with the training distribution.
- Strengthen privacy: retention limits, access control, aggregation thresholds, and a review of platform terms before any real collection.
- Package the system with Docker Compose and CI, and add a reproducible benchmark at 10,000 messages/s on more than one machine.

References. The project used the following public sources and documentation:

[1] E. Biswas, IMDb Review Dataset - ebD, Kaggle, CC0. https://www.kaggle.com/datasets/ebiswas/imdb-review-dataset (DOI 10.34740/KAGGLE/DSV/1836923).

[2] Apache Software Foundation, Apache Spark 3.5.9 Documentation. https://spark.apache.org/docs/3.5.9/

[3] Apache Software Foundation, Apache Kafka 4.3 Documentation. https://kafka.apache.org/documentation/

[4] Apache Spark MLlib, Feature Extractors and Classification. https://spark.apache.org/docs/3.5.9/ml-features.html

[5] Streamlit, Streamlit Documentation. https://docs.streamlit.io/

5. Team Member Review and Comment

The comments below summarise each member's documented contributions. Each member should confirm or revise their own row before submission.

Team photo: insert the official group photo before submission. No team photo was available in the project repository when this report was generated.

NAME	REVIEW and COMMENT
Vàng A Ký (Team Leader)	Coordinated the plan, WBS, and Action Plan; framed the topic and the Technology-for-Good scope; integrated the final report and presentations. Remaining: final rehearsal and submission check.
Nguyễn Đình Bằng	Set up Kafka 4.3.1 in KRaft mode and the three topics; built the replay producer and bad-event generator; measured ingestion throughput up to 384 messages/s. Remaining: multi-broker test.
Vàng Thị Dẳm	Wrote the shared Spark SQL text-cleaning module and the Structured Streaming job with explicit schema, event time, watermark, checkpoints, and DLQ; documented the scenario and operating guide. Remaining: crash/restart fault-tolerance test.
Dương Thị Hạnh	Verified the data source and CC0 licence; performed cleaning, deduplication, and the fixed split; trained and evaluated five models. Remaining: human-labelled validation of the neutral class.
Ngô Lương Nguyên	Built thirteen Spark SQL queries, fifteen figures, and insight notes; implemented the Streamlit dashboard and the model-trial program; prepared report figures. Remaining: dashboard screenshot and deployment guide.

6. Instructor Review and Comment

CATEGORY	SCORE	REVIEW and COMMENT
IDEA	__/10	
APPLICATION	__/30	
RESULT	__/30	
PROJECT MANAGEMENT	__/10	
PRESENTATION & REPORT	__/20	
TOTAL	__/100	